

Line-by-Line Mathematical Guide for HAETAE_Transcript.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

Transcript records and validity/extraction facts.

How To Read This Script

Read this file as the bridge from accepted signatures to structured mathematical objects used by the reductions.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	<code>require import AllCore List.</code>	Imports AllCore List. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	<code>require import HAETAE_Params HAETAE_Algebra HAETAE_ROM HAETAE_Assumptions.</code>	Imports HAETAE_Params HAETAE_Algebra HAETAE_ROM HAETAE_Assumptions. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
3	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4	<code>theory HAETAE_Transcript.</code>	Begins the namespace HAETAE_Transcript, grouping the declarations as one checked theory.
5	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6	<code>import HAETAE_Params.</code>	Opens HAETAE_Params so its names can be used without qualification in the following proof.
7	<code>import HAETAE_Algebra.</code>	Opens HAETAE_Algebra so its names can be used without qualification in the following proof.
8	<code>import HAETAE_ROM.</code>	Opens HAETAE_ROM so its names can be used without qualification in the following proof.
9	<code>import HAETAE_Assumptions.</code>	Opens HAETAE_Assumptions so its names can be used without qualification in the following proof.
10	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
11	<code>type transcript =</code>	Declares carrier set transcript. EasyCrypt will type-check all later functions and games over this mathematical domain.
12	<code>pkey * message * context * polyveck * poly * crh * challenge * signature.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
13	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
14	<code>op signature_commitment_highbits :</code>	Defines operation signature_commitment_highbits, a total mathematical function or abbreviation used by later declarations.

Line	EasyCrypt source	Mathematical reading
15	mode -> pkey -> message -> context -> signature -> polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
16	fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
17	sig_commitment_highbits sig.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
18	op signature_commitment_lowbits :	Defines operation signature_commitment_lowbits, a total mathematical function or abbreviation used by later declarations.
19	mode -> pkey -> message -> context -> signature -> poly =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
20	fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
21	sig_commitment_lowbits sig.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
22	op signature_message_hash :	Defines operation signature_message_hash, a total mathematical function or abbreviation used by later declarations.
23	mode -> pkey -> message -> context -> signature -> crh =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
24	fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
25	sig_message_hash sig.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
26	op signature_challenge :	Defines operation signature_challenge, a total mathematical function or abbreviation used by later declarations.
27	mode -> pkey -> message -> context -> signature -> challenge =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
28	fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
29	sig_challenge sig.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
30	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
31	op transcript_pk (tr : transcript) : pkey = tr.'1.	Defines operation transcript_pk, a total mathematical function or abbreviation used by later declarations.
32	op transcript_message (tr : transcript) : message = tr.'2.	Defines operation transcript_message, a total mathematical function or abbreviation used by later declarations.
33	op transcript_context (tr : transcript) : context = tr.'3.	Defines operation transcript_context, a total mathematical function or abbreviation used by later declarations.
34	op transcript_commitment_highbits (tr : transcript) : polyveck = tr.'4.	Defines operation transcript_commitment_highbits, a total mathematical function or abbreviation used by later declarations.
35	op transcript_commitment_lowbits (tr : transcript) : poly = tr.'5.	Defines operation transcript_commitment_lowbits, a total mathematical function or abbreviation used by later declarations.
36	op transcript_message_hash (tr : transcript) : crh = tr.'6.	Defines operation transcript_message_hash, a total mathematical function or abbreviation used by later declarations.
37	op transcript_challenge (tr : transcript) : challenge = tr.'7.	Defines operation transcript_challenge, a total mathematical function or abbreviation used by later declarations.
38	op transcript_signature (tr : transcript) : signature = tr.'8.	Defines operation transcript_signature, a total mathematical function or abbreviation used by later declarations.
39	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
40	op transcript_of_signature :	Defines operation transcript_of_signature, a total mathematical function or abbreviation used by later declarations.
41	mode -> pkey -> message -> context -> signature -> transcript =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
42	fun md pk m ctx sig =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
43	(pk, m, ctx,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
44	signature_commitment_highbits md pk m ctx sig,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
45	signature_commitment_lowbits md pk m ctx sig,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
46	signature_message_hash md pk m ctx sig,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
47	signature_challenge md pk m ctx sig,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
48	<code>sig).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
49	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
50	<code>op transcript_challenge_query (md : mode) (tr : transcript) : ro_query =</code>	Defines operation <code>transcript_challenge_query</code> , a total mathematical function or abbreviation used by later declarations.
51	<code>challenge_hash_query</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
52	<code>md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
53	<code>(transcript_commitment_highbits tr)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
54	<code>(transcript_commitment_lowbits tr)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
55	<code>(transcript_message_hash tr).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
56	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
57	<code>op transcript_signature_challenge_query (md : mode) (tr : transcript) :</code>	Defines operation <code>transcript_signature_challenge_query</code> , a total mathematical function or abbreviation used by later declarations.
58	<code>ro_query =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
59	<code>challenge_hash_query</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
60	<code>md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
61	<code>(sig_commitment_highbits (transcript_signature tr))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
62	<code>(sig_commitment_lowbits (transcript_signature tr))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
63	<code>(sig_message_hash (transcript_signature tr)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
64	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
65	<code>op transcript_verifies (md : mode) (tr : transcript) : bool =</code>	Defines predicate <code>transcript_verifies</code> , an event or well-formedness condition used in later implications and game proofs.
66	<code>verify_internal md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
67	<code>(transcript_pk tr)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
68	<code>(transcript_message tr)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
69	<code>(transcript_context tr)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
70	<code>(transcript_signature tr).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
71	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
72	<code>op transcript_challenge_matches (tr : transcript) (y : ro_output) : bool =</code>	Defines predicate <code>transcript_challenge_matches</code> , an event or well-formedness condition used in later implications and game proofs.
73	<code>ro_challenge_hash y = transcript_challenge tr.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
74	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
75	<code>op transcript_signature_challenge_matches</code>	Defines operation <code>transcript_signature_challenge_matches</code> , a total mathematical function or abbreviation used by later declarations.
76	<code>(tr : transcript) (y : ro_output) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
77	<code>ro_challenge_hash y = sig_challenge (transcript_signature tr).</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
78	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
79	<code>op transcript_fields_match_signature (tr : transcript) : bool =</code>	Defines predicate <code>transcript_fields_match_signature</code> , an event or well-formedness condition used in later implications and game proofs.
80	<code>transcript_commitment_highbits tr =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
81	<code>sig_commitment_highbits (transcript_signature tr) /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
82	<code>transcript_commitment_lowbits tr =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
83	<code>sig_commitment_lowbits (transcript_signature tr) /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
84	<code>transcript_message_hash tr =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
85	<code>sig_message_hash (transcript_signature tr) /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
86	<code>transcript_challenge tr =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
87	<code>sig_challenge (transcript_signature tr).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
88	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
89	<code>op transcript_valid (md : mode) (tr : transcript) : bool =</code>	Defines predicate <code>transcript_valid</code> , an event or well-formedness condition used in later implications and game proofs.
90	<code>transcript_verifies md tr /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
91	<code>valid_signature md (transcript_signature tr) /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
92	<code>transcript_fields_match_signature tr.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
93	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
94	<code>op transcript_public_equation (md : mode) (tr : transcript) : bool =</code>	Defines predicate <code>transcript_public_equation</code> , an event or well-formedness condition used in later implications and game proofs.
95	<code>verify_public_equation md (transcript_pk tr) (transcript_signature tr).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
96	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
97	<code>op same_fork_target (tr1 tr2 : transcript) : bool =</code>	Defines predicate <code>same_fork_target</code> , an event or well-formedness condition used in later implications and game proofs.
98	<code>transcript_pk tr1 = transcript_pk tr2 /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
99	<code>transcript_message tr1 = transcript_message tr2 /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
100	<code>transcript_context tr1 = transcript_context tr2 /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
101	<code>transcript_commitment_highbits tr1 = transcript_commitment_highbits tr2 /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
102	<code>transcript_commitment_lowbits tr1 = transcript_commitment_lowbits tr2 /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
103	<code>transcript_message_hash tr1 = transcript_message_hash tr2.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
104	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
105	<code>op distinct_fork_challenges (tr1 tr2 : transcript) : bool =</code>	Defines predicate <code>distinct_fork_challenges</code> , an event or well-formedness condition used in later implications and game proofs.
106	<code>transcript_challenge tr1 <> transcript_challenge tr2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
107	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
108	<code>op valid_forking_pair (md : mode) (tr1 tr2 : transcript) : bool =</code>	Defines predicate <code>valid_forking_pair</code> , an event or well-formedness condition used in later implications and game proofs.
109	<code>same_fork_target tr1 tr2 /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
110	<code>distinct_fork_challenges tr1 tr2 /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
111	<code>transcript_valid md tr1 /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
112	<code>transcript_valid md tr2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
113	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
114	op transcript_response (tr : transcript) : polyvecl =	Defines operation transcript_response, a total mathematical function or abbreviation used by later declarations.
115	sig_response (transcript_signature tr).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
116	op transcript_response_aux (tr : transcript) : polyveck =	Defines operation transcript_response_aux, a total mathematical function or abbreviation used by later declarations.
117	sig_response_aux (transcript_signature tr).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
118	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
119	op transcript_public_key_challenge_term (md : mode) (tr : transcript) :	Defines operation transcript_public_key_challenge_term, a total mathematical function or abbreviation used by later declarations.
120	polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
121	public_key_challenge_term md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
122	(transcript_pk tr)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
123	(sig_challenge (transcript_signature tr)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
124	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
125	op active_public_reconstruction (md : mode) (tr : transcript) : bool =	Defines predicate active_public_reconstruction, an event or well-formedness condition used in later implications and game proofs.
126	transcript_public_key_challenge_term md tr <> polyveck_zero md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
127	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
128	op inactive_public_reconstruction (md : mode) (tr : transcript) : bool =	Defines predicate inactive_public_reconstruction, an event or well-formedness condition used in later implications and game proofs.
129	transcript_public_key_challenge_term md tr = polyveck_zero md.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
130	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
131	op forking_difference_solution	Defines operation forking_difference_solution, a total mathematical function or abbreviation used by later declarations.
132	(md : mode) (tr1 tr2 : transcript) : bimodal_selftarget_msis_solution =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
133	polyvecl_sub md (transcript_response tr1) (transcript_response tr2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
134	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
135	op fork_response_aux_difference (md : mode) (tr1 tr2 : transcript) :	Defines operation fork_response_aux_difference, a total mathematical function or abbreviation used by later declarations.
136	polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
137	polyveck_sub md (transcript_response_aux tr1) (transcript_response_aux tr2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
138	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
139	op fork_public_challenge_relation (md : mode) (tr1 tr2 : transcript) : bool =	Defines predicate fork_public_challenge_relation, an event or well-formedness condition used in later implications and game proofs.
140	polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
141	(transcript_response_aux tr1)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
142	(transcript_public_key_challenge_term md tr1) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
143	polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
144	(transcript_response_aux tr2)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
145	(transcript_public_key_challenge_term md tr2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
146	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
147	<code>op fork_response_aux_relation (tr1 tr2 : transcript) : bool =</code>	Defines predicate <code>fork_response_aux_relation</code> , an event or well-formedness condition used in later implications and game proofs.
148	<code>transcript_response_aux tr1 = transcript_response_aux tr2.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
149	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
150	<code>op fork_left_active_public_challenge_relation</code>	Defines operation <code>fork_left_active_public_challenge_relation</code> , a total mathematical function or abbreviation used by later declarations.
151	<code>(md : mode) (tr1 tr2 : transcript) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
152	<code>polyveck_add md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
153	<code>(transcript_response_aux tr1)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
154	<code>(transcript_public_key_challenge_term md tr1) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
155	<code>transcript_response_aux tr2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
156	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
157	<code>op fork_right_active_public_challenge_relation</code>	Defines operation <code>fork_right_active_public_challenge_relation</code> , a total mathematical function or abbreviation used by later declarations.
158	<code>(md : mode) (tr1 tr2 : transcript) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
159	<code>transcript_response_aux tr1 =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
160	<code>polyveck_add md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
161	<code>(transcript_response_aux tr2)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
162	<code>(transcript_public_key_challenge_term md tr2).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
163	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
164	<code>op fork_public_reconstruction_cases</code>	Defines operation <code>fork_public_reconstruction_cases</code> , a total mathematical function or abbreviation used by later declarations.
165	<code>(md : mode) (tr1 tr2 : transcript) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
166	<code>fork_public_challenge_relation md tr1 tr2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
167	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
168	<code>op fork_matrix_source (tr1 tr2 : transcript) : byte list =</code>	Defines operation <code>fork_matrix_source</code> , a total mathematical function or abbreviation used by later declarations.
169	<code>encode_pkey (transcript_pk tr1) ++</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
170	<code>transcript_context tr1 ++</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
171	<code>transcript_message tr1 ++</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
172	<code>encode_polyveck (transcript_commitment_highbits tr1) ++</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
173	<code>encode_poly (transcript_commitment_lowbits tr1) ++</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
174	<code>transcript_message_hash tr1 ++</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
175	<code>encode_poly (transcript_challenge tr1) ++</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
176	<code>encode_poly (transcript_challenge tr2).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
177	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
178	<code>op fork_module_matrix (md : mode) (tr1 tr2 : transcript) : matrix =</code>	Defines operation <code>fork_module_matrix</code> , a total mathematical function or abbreviation used by later declarations.
179	<code>mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
180	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
181	deterministic_unit_polyvec1 md (53 + i) (fork_matrix_source tr1 tr2))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
182	(mode_k md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
183	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
184	op fork_module_target (md : mode) (tr1 tr2 : transcript) : polyveck =	Defines operation fork_module_target, a total mathematical function or abbreviation used by later declarations.
185	matrix_vec_mul md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
186	(fork_module_matrix md tr1 tr2)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
187	(forking_difference_solution md tr1 tr2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
188	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
189	op bimodal_selftarget_msis_instance_of_fork :	Defines operation bimodal_selftarget_msis_instance_of_fork, a total mathematical function or abbreviation used by later declarations.
190	mode -> transcript -> transcript -> bimodal_selftarget_msis_instance =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
191	fun md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
192	(fork_module_matrix md tr1 tr2, fork_module_target md tr1 tr2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
193	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
194	op extract_bimodal_selftarget_msis_solution :	Defines operation extract_bimodal_selftarget_msis_solution, a total mathematical function or abbreviation used by later declarations.
195	mode -> transcript -> transcript ->	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
196	bimodal_selftarget_msis_solution option =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
197	fun md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
198	if valid_forking_pair md tr1 tr2	Splits proof obligations according to a conditional branch in the program or formula.
199	then Some (forking_difference_solution md tr1 tr2)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
200	else None.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
201	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
202	op module_sis_instance_of_fork :	Defines operation module_sis_instance_of_fork, a total mathematical function or abbreviation used by later declarations.
203	mode -> transcript -> transcript -> module_sis_instance =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
204	fun md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
205	bimodal_to_module_sis_instance md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
206	(bimodal_selftarget_msis_instance_of_fork md tr1 tr2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
207	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
208	op extract_module_sis_solution :	Defines operation extract_module_sis_solution, a total mathematical function or abbreviation used by later declarations.
209	mode -> transcript -> transcript -> module_sis_solution option =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
210	fun md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
211	let x = bimodal_selftarget_msis_instance_of_fork md tr1 tr2 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
212	let sol = extract_bimodal_selftarget_msis_solution md tr1 tr2 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
213	<code>if sol = None then None</code>	Splits proof obligations according to a conditional branch in the program or formula.
214	<code>else Some (bimodal_to_module_sis_solution md x (oget sol)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
215	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
216	<code>op bimodal_extraction_success (md : mode) (tr1 tr2 : transcript) : bool =</code>	Defines predicate <code>bimodal_extraction_success</code> , an event or well-formedness condition used in later implications and game proofs.
217	<code>extract_bimodal_selftarget_msis_solution md tr1 tr2 <> None /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
218	<code>bimodal_selftarget_msis_valid md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
219	<code>(bimodal_selftarget_msis_instance_of_fork md tr1 tr2)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
220	<code>(oget (extract_bimodal_selftarget_msis_solution md tr1 tr2)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
221	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
222	<code>lemma extract_bimodal_forking_some md tr1 tr2 :</code>	States checked lemma <code>extract_bimodal_forking_some</code> . The following proof script must derive this proposition from prior definitions and lemmas.
223	<code>valid_forking_pair md tr1 tr2 =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
224	<code>extract_bimodal_selftarget_msis_solution md tr1 tr2 =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
225	<code>Some (forking_difference_solution md tr1 tr2).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
226	<code>proof.</code>	Begins the proof script for the preceding proposition.
227	<code>rewrite /extract_bimodal_selftarget_msis_solution.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
228	<code>by case: (valid_forking_pair md tr1 tr2).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
229	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
230	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
231	<code>lemma extract_bimodal_forking_nonempty md tr1 tr2 :</code>	States checked lemma <code>extract_bimodal_forking_nonempty</code> . The following proof script must derive this proposition from prior definitions and lemmas.
232	<code>valid_forking_pair md tr1 tr2 =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
233	<code>extract_bimodal_selftarget_msis_solution md tr1 tr2 <> None.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
234	<code>proof.</code>	Begins the proof script for the preceding proposition.
235	<code>move=> valid_pair.</code>	Introduces universally quantified variables or hypotheses into the proof context.
236	<code>rewrite (extract_bimodal_forking_some md tr1 tr2 valid_pair).</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
237	<code>by case: (Some (forking_difference_solution md tr1 tr2) = None).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
238	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
239	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
240	<code>lemma transcript_fields_match_signature_challenge_query md tr :</code>	States checked lemma <code>transcript_fields_match_signature_challenge_query</code> . The following proof script must derive this proposition from prior definitions and lemmas.
241	<code>transcript_fields_match_signature tr =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
242	<code>transcript_challenge_query md tr =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
243	<code>transcript_signature_challenge_query md tr.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
244	<code>proof.</code>	Begins the proof script for the preceding proposition.
245	<code>rewrite /transcript_fields_match_signature</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
246	<code>/transcript_challenge_query</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
247	<code>/transcript_signature_challenge_query.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
248	<code>move=> [highE rest].</code>	Introduces universally quantified variables or hypotheses into the proof context.
249	<code>case: rest => lowE rest.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
250	<code>case: rest => muE _.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
251	<code>by rewrite highE lowE muE.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
252	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
253	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
254	<code>lemma transcript_fields_match_signature_challenge_matches tr y :</code>	States checked lemma <code>transcript_fields_match_signature_challenge_matches</code> . The following proof script must derive this proposition from prior definitions and lemmas.
255	<code>transcript_fields_match_signature tr =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
256	<code>transcript_challenge_matches tr y =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
257	<code>transcript_signature_challenge_matches tr y.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
258	<code>proof.</code>	Begins the proof script for the preceding proposition.
259	<code>rewrite /transcript_fields_match_signature</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
260	<code>/transcript_challenge_matches</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
261	<code>/transcript_signature_challenge_matches.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
262	<code>move=> [_ rest].</code>	Introduces universally quantified variables or hypotheses into the proof context.
263	<code>case: rest => _ rest.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
264	<code>case: rest => _ chE.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
265	<code>by rewrite chE.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
266	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
267	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
268	<code>lemma transcript_valid_signature_challenge_query md tr :</code>	States checked lemma <code>transcript_valid_signature_challenge_query</code> . The following proof script must derive this proposition from prior definitions and lemmas.
269	<code>transcript_valid md tr =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
270	<code>transcript_challenge_query md tr =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
271	<code>transcript_signature_challenge_query md tr.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
272	<code>proof.</code>	Begins the proof script for the preceding proposition.
273	<code>rewrite /transcript_valid.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
274	<code>move=> [_ [_ fields]].</code>	Introduces universally quantified variables or hypotheses into the proof context.
275	<code>by apply transcript_fields_match_signature_challenge_query.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
276	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
277	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
278	<code>lemma transcript_valid_signature_challenge_matches md tr y :</code>	States checked lemma <code>transcript_valid_signature_challenge_matches</code> . The following proof script must derive this proposition from prior definitions and lemmas.
279	<code>transcript_valid md tr =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
280	<code>transcript_challenge_matches tr y =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
281	<code>transcript_signature_challenge_matches tr y.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
282	proof.	Begins the proof script for the preceding proposition.
283	rewrite /transcript_valid.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
284	move=> [_ [_ fields]].	Introduces universally quantified variables or hypotheses into the proof context.
285	by apply transcript_fields_match_signature_challenge_matches.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
286	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
287	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
288	lemma transcript_valid_signature_challengeE md tr :	States checked lemma transcript_valid_signature_challengeE. The following proof script must derive this proposition from prior definitions and lemmas.
289	transcript_valid md tr =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
290	transcript_challenge tr = sig_challenge (transcript_signature tr).	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
291	proof.	Begins the proof script for the preceding proposition.
292	rewrite /transcript_valid /transcript_fields_match_signature.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
293	move=> [_ [_ _ rest]]].	Introduces universally quantified variables or hypotheses into the proof context.
294	case: rest => _ rest.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
295	case: rest => _ chE.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
296	by apply chE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
297	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
298	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
299	lemma transcript_valid_public_equation md tr :	States checked lemma transcript_valid_public_equation. The following proof script must derive this proposition from prior definitions and lemmas.
300	transcript_valid md tr =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
301	transcript_public_equation md tr.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
302	proof.	Begins the proof script for the preceding proposition.
303	rewrite /transcript_valid /transcript_verifies	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
304	/transcript_public_equation /verify_internal.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
305	move=> h.	Introduces universally quantified variables or hypotheses into the proof context.
306	case: h => hver _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
307	case: hver => _ htail.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
308	case: htail => _ hpubnorm.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
309	case: hpubnorm => pub _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
310	by apply pub.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
311	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
312	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
313	lemma transcript_valid_commitment_reconstruction md tr :	States checked lemma transcript_valid_commitment_reconstruction. The following proof script must derive this proposition from prior definitions and lemmas.
314	transcript_valid md tr =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
315	transcript_commitment_highbits tr =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
316	verify_reconstructed_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
317	(transcript_pk tr)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
318	(transcript_signature tr).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
319	proof.	Begins the proof script for the preceding proposition.
320	move=> valid.	Introduces universally quantified variables or hypotheses into the proof context.
321	have pub := transcript_valid_public_equation md tr valid.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
322	move: pub.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
323	rewrite /transcript_public_equation /verify_public_equation.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
324	move=> pubE.	Introduces universally quantified variables or hypotheses into the proof context.
325	move: valid.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
326	rewrite /transcript_valid /transcript_fields_match_signature.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
327	case=> _ rest.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
328	case: rest => _ fields.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
329	case: fields => highE _.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
330	by rewrite highE pubE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
331	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
332	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
333	lemma valid_forking_pair_public_equations md tr1 tr2 :	States checked lemma valid_forking_pair_public_equations. The following proof script must derive this proposition from prior definitions and lemmas.
334	valid_forking_pair md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
335	transcript_public_equation md tr1 /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
336	transcript_public_equation md tr2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
337	proof.	Begins the proof script for the preceding proposition.
338	rewrite /valid_forking_pair.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
339	move=> [_ [_ [valid1 valid2]]].	Introduces universally quantified variables or hypotheses into the proof context.
340	split.	Splits a conjunction or structured goal into independent proof obligations.
341	+ by apply transcript_valid_public_equation.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
342	+ by apply transcript_valid_public_equation.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
343	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
344	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
345	lemma same_fork_target_challenge_query md tr1 tr2 :	States checked lemma same_fork_target_challenge_query. The following proof script must derive this proposition from prior definitions and lemmas.
346	same_fork_target tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
347	transcript_challenge_query md tr1 = transcript_challenge_query md tr2.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
348	proof.	Begins the proof script for the preceding proposition.
349	rewrite /same_fork_target /transcript_challenge_query.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
350	move=> [_ [_ [_ [highE [lowE muE]]]]].	Introduces universally quantified variables or hypotheses into the proof context.
351	by rewrite highE lowE muE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
352	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
353	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
354	lemma valid_forking_pair_challenge_query md tr1 tr2 :	States checked lemma valid_forking_pair_challenge_query. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
355	<code>valid_forking_pair md tr1 tr2 =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
356	<code>transcript_challenge_query md tr1 = transcript_challenge_query md tr2.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
357	<code>proof.</code>	Begins the proof script for the preceding proposition.
358	<code>rewrite /valid_forking_pair.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
359	<code>move=> [same _].</code>	Introduces universally quantified variables or hypotheses into the proof context.
360	<code>by apply same_fork_target_challenge_query.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
361	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
362	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
363	<code>lemma valid_forking_pair_signature_challenge_query md tr1 tr2 :</code>	States checked lemma <code>valid_forking_pair_signature_challenge_query</code> . The following proof script must derive this proposition from prior definitions and lemmas.
364	<code>valid_forking_pair md tr1 tr2 =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
365	<code>transcript_signature_challenge_query md tr1 =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
366	<code>transcript_signature_challenge_query md tr2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
367	<code>proof.</code>	Begins the proof script for the preceding proposition.
368	<code>move=> valid_pair.</code>	Introduces universally quantified variables or hypotheses into the proof context.
369	<code>have queryE := valid_forking_pair_challenge_query md tr1 tr2 valid_pair.</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
370	<code>move: valid_pair.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
371	<code>rewrite /valid_forking_pair.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
372	<code>move=> [_ [_ [valid1 valid2]]].</code>	Introduces universally quantified variables or hypotheses into the proof context.
373	<code>rewrite -(transcript_valid_signature_challenge_query md tr1 valid1).</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
374	<code>rewrite -(transcript_valid_signature_challenge_query md tr2 valid2).</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
375	<code>by apply queryE.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
376	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
377	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
378	<code>lemma valid_forking_pair_distinct_signature_challenges md tr1 tr2 :</code>	States checked lemma <code>valid_forking_pair_distinct_signature_challenges</code> . The following proof script must derive this proposition from prior definitions and lemmas.
379	<code>valid_forking_pair md tr1 tr2 =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
380	<code>sig_challenge (transcript_signature tr1) <></code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
381	<code>sig_challenge (transcript_signature tr2).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
382	<code>proof.</code>	Begins the proof script for the preceding proposition.
383	<code>rewrite /valid_forking_pair.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
384	<code>move=> [_ [distinct [valid1 valid2]]].</code>	Introduces universally quantified variables or hypotheses into the proof context.
385	<code>have ch1E := transcript_valid_signature_challengeE md tr1 valid1.</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
386	<code>have ch2E := transcript_valid_signature_challengeE md tr2 valid2.</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
387	<code>move: distinct.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
388	<code>rewrite /distinct_fork_challenges ch1E ch2E.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
389	<code>by [].</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
390	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
391	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
392	lemma same_fork_target_commitment_highbits tr1 tr2 :	States checked lemma same_fork_target_commitment_highbits. The following proof script must derive this proposition from prior definitions and lemmas.
393	same_fork_target tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
394	transcript_commitment_highbits tr1 = transcript_commitment_highbits tr2.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
395	proof. by rewrite /same_fork_target; move=> [_ [_ [_ [highE _]]]]. qed.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
396	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
397	lemma valid_forking_pair_commitment_highbits md tr1 tr2 :	States checked lemma valid_forking_pair_commitment_highbits. The following proof script must derive this proposition from prior definitions and lemmas.
398	valid_forking_pair md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
399	transcript_commitment_highbits tr1 = transcript_commitment_highbits tr2.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
400	proof.	Begins the proof script for the preceding proposition.
401	rewrite /valid_forking_pair.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
402	move=> [same _].	Introduces universally quantified variables or hypotheses into the proof context.
403	by apply same_fork_target_commitment_highbits.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
404	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
405	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
406	lemma valid_forking_pair_reconstructed_highbits_equal md tr1 tr2 :	States checked lemma valid_forking_pair_reconstructed_highbits_equal. The following proof script must derive this proposition from prior definitions and lemmas.
407	valid_forking_pair md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
408	verify_reconstructed_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
409	(transcript_pk tr1) (transcript_signature tr1) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
410	verify_reconstructed_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
411	(transcript_pk tr2) (transcript_signature tr2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
412	proof.	Begins the proof script for the preceding proposition.
413	move=> valid_pair.	Introduces universally quantified variables or hypotheses into the proof context.
414	have highE := valid_forking_pair_commitment_highbits md tr1 tr2 valid_pair.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
415	move: valid_pair.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
416	rewrite /valid_forking_pair.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
417	move=> [_ [_ [valid1 valid2]]].	Introduces universally quantified variables or hypotheses into the proof context.
418	have rec1 := transcript_valid_commitment_reconstruction md tr1 valid1.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
419	have rec2 := transcript_valid_commitment_reconstruction md tr2 valid2.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
420	by rewrite -rec1 -rec2.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
421	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
422	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
423	lemma transcript_reconstructionE md tr :	States checked lemma transcript_reconstructionE. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
424	verify_reconstructed_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
425	(transcript_pk tr) (transcript_signature tr) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
426	polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
427	(transcript_response_aux tr)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
428	(transcript_public_key_challenge_term md tr).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
429	proof.	Begins the proof script for the preceding proposition.
430	by rewrite /verify_reconstructed_highbits	Discharges the current goal in one step using the cited simplification, lemma, or automation.
431	/transcript_response_aux	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
432	/transcript_public_key_challenge_term	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
433	/reconstructed_highbits.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
434	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
435	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
436	lemma transcript_active_reconstructionE md tr :	States checked lemma transcript_active_reconstructionE. The following proof script must derive this proposition from prior definitions and lemmas.
437	active_public_reconstruction md tr =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
438	verify_reconstructed_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
439	(transcript_pk tr) (transcript_signature tr) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
440	polyveck_add md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
441	(transcript_response_aux tr)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
442	(transcript_public_key_challenge_term md tr).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
443	proof.	Begins the proof script for the preceding proposition.
444	move=> _.	Introduces universally quantified variables or hypotheses into the proof context.
445	by apply transcript_reconstructionE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
446	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
447	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
448	lemma valid_forking_pair_public_challenge_relation md tr1 tr2 :	States checked lemma valid_forking_pair_public_challenge_relation. The following proof script must derive this proposition from prior definitions and lemmas.
449	valid_forking_pair md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
450	fork_public_challenge_relation md tr1 tr2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
451	proof.	Begins the proof script for the preceding proposition.
452	move=> valid_pair.	Introduces universally quantified variables or hypotheses into the proof context.
453	have recE :=	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
454	valid_forking_pair_reconstructed_highbits_equal md tr1 tr2 valid_pair.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
455	rewrite /fork_public_challenge_relation.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
456	rewrite -(transcript_reconstructionE md tr1).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
457	rewrite -(transcript_reconstructionE md tr2).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
458	by apply recE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
459	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
460	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
461	lemma valid_forking_pair_public_reconstruction_cases md tr1 tr2 :	States checked lemma <code>valid_forking_pair_public_reconstruction_cases</code> . The following proof script must derive this proposition from prior definitions and lemmas.
462	valid_forking_pair md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
463	fork_public_reconstruction_cases md tr1 tr2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
464	proof.	Begins the proof script for the preceding proposition.
465	move=> valid_pair.	Introduces universally quantified variables or hypotheses into the proof context.
466	rewrite /fork_public_reconstruction_cases.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
467	by apply valid_forking_pair_public_challenge_relation.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
468	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
469	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
470	op extraction_success (md : mode) (tr1 tr2 : transcript) : bool =	Defines predicate <code>extraction_success</code> , an event or well-formedness condition used in later implications and game proofs.
471	extract_module_sis_solution md tr1 tr2 <> None /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
472	module_sis_valid md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
473	(module_sis_instance_of_fork md tr1 tr2)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
474	(oget (extract_module_sis_solution md tr1 tr2)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
475	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
476	op forking_extraction_obligation (md : mode) : bool =	Defines predicate <code>forking_extraction_obligation</code> , an event or well-formedness condition used in later implications and game proofs.
477	forall (tr1 tr2 : transcript),	Part of a universal mathematical condition that must hold for all listed values.
478	valid_forking_pair md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
479	bimodal_extraction_success md tr1 tr2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
480	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
481	op fork_public_reconstruction_obligation (md : mode) : bool =	Defines predicate <code>fork_public_reconstruction_obligation</code> , an event or well-formedness condition used in later implications and game proofs.
482	forall (tr1 tr2 : transcript),	Part of a universal mathematical condition that must hold for all listed values.
483	valid_forking_pair md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
484	fork_public_reconstruction_cases md tr1 tr2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
485	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
486	lemma forking_difference_solution_wf md tr1 tr2 :	States checked lemma <code>forking_difference_solution_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
487	module_sis_solution_wf md (forking_difference_solution md tr1 tr2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
488	proof.	Begins the proof script for the preceding proposition.
489	by rewrite /module_sis_solution_wf /forking_difference_solution	Discharges the current goal in one step using the cited simplification, lemma, or automation.
490	polyvecl_sub_wf.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
491	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
492	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
493	lemma fork_module_matrix_rows_wf md tr1 tr2 :	States checked lemma <code>fork_module_matrix_rows_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
494	<code>all (polyvecl_wf md) (fork_module_matrix md tr1 tr2).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
495	<code>proof.</code>	Begins the proof script for the preceding proposition.
496	<code>rewrite /fork_module_matrix.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
497	<code>apply/allP=> row /mkseqP [i [_ ->]].</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
498	<code>by apply deterministic_unit_polyvecl_wf.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
499	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
500	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
501	<code>lemma fork_module_matrix_size md tr1 tr2 :</code>	States checked lemma <code>fork_module_matrix_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
502	<code>size (fork_module_matrix md tr1 tr2) = mode_k md.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
503	<code>proof. by rewrite /fork_module_matrix size_mkseq; case md. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
504	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
505	<code>lemma bimodal_forking_solution_valid md tr1 tr2 :</code>	States checked lemma <code>bimodal_forking_solution_valid</code> . The following proof script must derive this proposition from prior definitions and lemmas.
506	<code>bimodal_selftarget_msis_valid md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
507	<code>(bimodal_selftarget_msis_instance_of_fork md tr1 tr2)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
508	<code>(forking_difference_solution md tr1 tr2).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
509	<code>proof.</code>	Begins the proof script for the preceding proposition.
510	<code>rewrite /bimodal_selftarget_msis_valid /module_sis_valid.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
511	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
512	<code>+ by apply forking_difference_solution_wf.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
513	<code>by rewrite /bimodal_selftarget_msis_instance_of_fork /fork_module_target /=.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
514	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
515	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
516	<code>lemma forking_extraction_obligation_holds md :</code>	States checked lemma <code>forking_extraction_obligation_holds</code> . The following proof script must derive this proposition from prior definitions and lemmas.
517	<code>forking_extraction_obligation md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
518	<code>proof.</code>	Begins the proof script for the preceding proposition.
519	<code>rewrite /forking_extraction_obligation /bimodal_extraction_success.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
520	<code>move=> tr1 tr2 valid_pair.</code>	Introduces universally quantified variables or hypotheses into the proof context.
521	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
522	<code>+ by apply extract_bimodal_forking_nonempty.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
523	<code>+ rewrite (extract_bimodal_forking_some md tr1 tr2 valid_pair) /=.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
524	<code>by apply bimodal_forking_solution_valid.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
525	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
526	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
527	<code>lemma fork_public_reconstruction_obligation_holds md :</code>	States checked lemma <code>fork_public_reconstruction_obligation_holds</code> . The following proof script must derive this proposition from prior definitions and lemmas.
528	<code>fork_public_reconstruction_obligation md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
529	<code>proof.</code>	Begins the proof script for the preceding proposition.
530	<code>rewrite /fork_public_reconstruction_obligation.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
531	<code>move=> tr1 tr2 valid_pair.</code>	Introduces universally quantified variables or hypotheses into the proof context.
532	<code>by apply valid_forking_pair_public_reconstruction_cases.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
533	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
534	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
535	<code>op bimodal_to_module_sis_lift_obligation (md : mode) : bool =</code>	Defines predicate <code>bimodal_to_module_sis_lift_obligation</code> , an event or well-formedness condition used in later implications and game proofs.
536	<code>forall (x : bimodal_selftarget_msis_instance)</code>	Part of a universal mathematical condition that must hold for all listed values.
537	<code>(sol : bimodal_selftarget_msis_solution),</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
538	<code>bimodal_selftarget_msis_valid md x sol =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
539	<code>module_sis_valid md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
540	<code>(bimodal_to_module_sis_instance md x)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
541	<code>(bimodal_to_module_sis_solution md x sol).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
542	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
543	<code>lemma bimodal_to_module_sis_lift_obligation_holds md :</code>	States checked lemma <code>bimodal_to_module_sis_lift_obligation_holds</code> . The following proof script must derive this proposition from prior definitions and lemmas.
544	<code>bimodal_to_module_sis_lift_obligation md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
545	<code>proof.</code>	Begins the proof script for the preceding proposition.
546	<code>rewrite /bimodal_to_module_sis_lift_obligation.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
547	<code>move=> x sol valid.</code>	Introduces universally quantified variables or hypotheses into the proof context.
548	<code>by apply bimodal_to_module_sis_valid.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
549	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
550	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
551	<code>op special_soundness_obligation (md : mode) : bool =</code>	Defines predicate <code>special_soundness_obligation</code> , an event or well-formedness condition used in later implications and game proofs.
552	<code>forall (tr1 tr2 : transcript),</code>	Part of a universal mathematical condition that must hold for all listed values.
553	<code>valid_forking_pair md tr1 tr2 =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
554	<code>extraction_success md tr1 tr2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
555	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
556	<code>op special_soundness_with_public_reconstruction_obligation (md : mode) : bool =</code>	Defines predicate <code>special_soundness_with_public_reconstruction_obligation</code> , an event or well-formedness condition used in later implications and game proofs.
557	<code>forall (tr1 tr2 : transcript),</code>	Part of a universal mathematical condition that must hold for all listed values.
558	<code>valid_forking_pair md tr1 tr2 =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
559	<code>fork_public_reconstruction_cases md tr1 tr2 /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
560	<code>extraction_success md tr1 tr2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
561	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
562	<code>lemma module_sis_extraction_from_bimodal md tr1 tr2 :</code>	States checked lemma <code>module_sis_extraction_from_bimodal</code> . The following proof script must derive this proposition from prior definitions and lemmas.
563	<code>bimodal_extraction_success md tr1 tr2 =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
564	<code>bimodal_to_module_sis_lift_obligation md =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
565	extraction_success md tr1 tr2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
566	proof.	Begins the proof script for the preceding proposition.
567	rewrite /bimodal_extraction_success /bimodal_to_module_sis_lift_obligation	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
568	/extraction_success /extract_module_sis_solution	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
569	/module_sis_instance_of_fork.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
570	move=> [nz valid] lift.	Introduces universally quantified variables or hypotheses into the proof context.
571	split.	Splits a conjunction or structured goal into independent proof obligations.
572	+ rewrite /extract_module_sis_solution /= iff.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
573	+ by apply nz.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
574	by rewrite /.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
575	rewrite /extract_module_sis_solution /= iff.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
576	+ by apply nz.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
577	apply (lift (bimodal_selftarget_msis_instance_of_fork md tr1 tr2))	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
578	(oget (extract_bimodal_selftarget_msis_solution md tr1 tr2))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
579	by apply valid.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
580	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
581	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
582	lemma special_soundness_from_bimodal_lift md :	States checked lemma special_soundness_from_bimodal_lift. The following proof script must derive this proposition from prior definitions and lemmas.
583	forking_extraction_obligation md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
584	bimodal_to_module_sis_lift_obligation md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
585	special_soundness_obligation md.	Performs symbolic program simplification, exposing assignments and state updates.
586	proof.	Begins the proof script for the preceding proposition.
587	rewrite /forking_extraction_obligation /special_soundness_obligation.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
588	move=> fork lift tr1 tr2 valid_pair.	Introduces universally quantified variables or hypotheses into the proof context.
589	by apply module_sis_extraction_from_bimodal; [apply fork apply lift].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
590	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
591	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
592	lemma special_soundness_with_public_reconstruction_from_bimodal_lift md :	States checked lemma special_soundness_with_public_reconstruction_from_bimodal_lift. The following proof script must derive this proposition from prior definitions and lemmas.
593	fork_public_reconstruction_obligation md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
594	forking_extraction_obligation md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
595	bimodal_to_module_sis_lift_obligation md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
596	special_soundness_with_public_reconstruction_obligation md.	Performs symbolic program simplification, exposing assignments and state updates.
597	proof.	Begins the proof script for the preceding proposition.
598	rewrite /fork_public_reconstruction_obligation	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
599	/forking_extraction_obligation	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
600	/special_soundness_with_public_reconstruction_obligation.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
601	move=> public fork lift tr1 tr2 valid_pair.	Introduces universally quantified variables or hypotheses into the proof context.
602	split.	Splits a conjunction or structured goal into independent proof obligations.
603	+ by apply public.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
604	by apply module_sis_extraction_from_bimodal; [apply fork apply lift].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
605	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
606	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
607	op transcript_from_honest_signing	Defines operation transcript_from_honest_signing, a total mathematical function or abbreviation used by later declarations.
608	(md : mode) (sk : skey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
609	(coins : random_coins) : transcript =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
610	transcript_of_signature md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
611	(public_key_of_secret md sk)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
612	m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
613	(sign_internal md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
614	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
615	lemma honest_transcript_verifies md sk m ctx coins :	States checked lemma honest_transcript_verifies. The following proof script must derive this proposition from prior definitions and lemmas.
616	transcript_verifies md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
617	(transcript_from_honest_signing md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
618	proof.	Begins the proof script for the preceding proposition.
619	rewrite /transcript_from_honest_signing /transcript_verifies	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
620	/transcript_of_signature /transcript_pk /transcript_message	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
621	/transcript_context /transcript_signature /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
622	by apply verify_internal_sign_internal.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
623	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
624	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
625	lemma transcript_of_signature_fields md pk m ctx sig :	States checked lemma transcript_of_signature_fields. The following proof script must derive this proposition from prior definitions and lemmas.
626	transcript_commitment_highbits	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
627	(transcript_of_signature md pk m ctx sig) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
628	sig_commitment_highbits sig /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
629	transcript_commitment_lowbits	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
630	(transcript_of_signature md pk m ctx sig) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
631	sig_commitment_lowbits sig /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
632	transcript_message_hash	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
633	(transcript_of_signature md pk m ctx sig) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
634	sig_message_hash sig /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
635	<code>transcript_challenge</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
636	<code>(transcript_of_signature md pk m ctx sig) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
637	<code>sig_challenge sig.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
638	<code>proof. by rewrite /transcript_of_signature /=. qed.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
639	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
640	<code>lemma transcript_of_signature_matches_signature md pk m ctx sig :</code>	States checked lemma <code>transcript_of_signature_matches_signature</code> . The following proof script must derive this proposition from prior definitions and lemmas.
641	<code>transcript_fields_match_signature</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
642	<code>(transcript_of_signature md pk m ctx sig).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
643	<code>proof.</code>	Begins the proof script for the preceding proposition.
644	<code>by rewrite /transcript_fields_match_signature /transcript_of_signature /=.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
645	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
646	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
647	<code>lemma honest_transcript_challenge md sk m ctx coins :</code>	States checked lemma <code>honest_transcript_challenge</code> . The following proof script must derive this proposition from prior definitions and lemmas.
648	<code>transcript_challenge</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
649	<code>(transcript_from_honest_signing md sk m ctx coins) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
650	<code>challenge_hash</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
651	<code>md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
652	<code>(commitment_highbits md sk m ctx coins)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
653	<code>(commitment_lowbits md sk m ctx coins)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
654	<code>(message_hash (public_key_of_secret md sk) ctx m).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
655	<code>proof.</code>	Begins the proof script for the preceding proposition.
656	<code>by rewrite /transcript_from_honest_signing /transcript_of_signature</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
657	<code>/signature_challenge /sig_challenge /sign_internal /=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
658	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
659	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
660	<code>end HAETAE_Transcript.</code>	Closes the current theory or module block.